

Domain-Specific Text Analysis & Retrieval System for Indian Legal Judgments

Individual Task Observations & Comprehensive Viva Defense Guide

NLP Assessment 1 · Seventh Semester · School of Advanced Computing, Vidyashilp University

Member	Assigned Engineering Module	Key Contribution & Technical Defense Focus
■ Brunda	Ingestion, Text Cleaning & Baseline Stemming	Ingested 25 court PDFs (257k tokens); conservative cleaning preserving ■ and sub-clauses; empirical benchmark of Porter vs. Snowball vs. Lancaster.
■ Shreerenu	Stopwords, Baseline Pipeline A & Word Inverted Index	Engineered 8-term Protected Stopword Policy (49.9% compression); built word-level positional inverted index; mathematically proved over-stemming collisions (38.5% precision on 'executive').
■ Lavanya	Custom Legal Regex, ML POS Classifier & Legal NER	Engineered Custom Legal Regex Tokenizer (atomic citations); evaluated Penn Treebank failures; trained Supervised Logistic Regression POS Classifier (76.7% acc, 0.705 F1); Legal NER.
■ Raju	Contextual Lemmatization, Dual Index, IR Engine & Evaluation	Engineered POS contextual lemmatization (100% precision on collisions); built Dual-Level Atomic Positional Index (3.5× faster); Boolean & TF-IDF Cosine IR engine; 15-query evaluation win (+8.9% precision, +5.7% F1); Streamlit GUI.

PART 1: INDIVIDUAL TASK OBSERVATIONS & EMPIRICAL FINDINGS

■ Brunda: Observations on Ingestion, Cleaning & Stemming

1. OCR Line-Break Hyphenation: Ingested 25 verified court judgments (257,849 tokens, 8,552 sentences). Court PDFs frequently contain broken words across line boundaries (e.g., crimi-nnal). Without proactive regex de-hyphenation, vocabulary counts artificially inflate, and queries for 'criminal' fail to match.

2. Aggressive Cleaning Flaws: Aggressive cleaning strips all non-alphanumerics, destroying Indian Rupee markers (■50,000/- -> 50 000) and deleting statutory sub-clause brackets (Section 376(2)(i) IPC -> section 376 2 i ipc). Deleting brackets destroys the aggravated statutory child rape threshold under the POCSO Act. Conservative cleaning preserves 100% of these statutory boundaries.

3. Stemmer Evaluation (Why Porter over Snowball & Lancaster?): Lancaster uses hyper-aggressive rules that destroy legal semantics: legal -> leg, custody -> cust, bailment -> bail (conflating contracts with criminal bail), and appellants -> appl. On our 257k tokens, Porter and Snowball produce 98.4% identical roots, and both fail on the exact same legal collisions (execution/executive both become execut). Porter was selected because it is the canonical, universally recognized baseline in IR benchmark literature (Manning et al., 2008), proving that suffix stripping itself is fundamentally inadequate.

■ Shreerenu: Observations on Stopwords, Pipeline A & Indexing

- 1. Liability Inversion:** Standard stopwords lists discard negation words. In criminal law, stripping these words causes catastrophic inversion: *'applicant is not guilty'* becomes ['applicant', 'guilty'], and *'no grounds for detention'* becomes ['grounds', 'detention']. Queries for acquittal retrieve convictions, collapsing precision to 0%.
- 2. 8-Term Protected Whitelist:** Whitelisted 8 critical terms: not, no, never, without, unless, except, until, against. This achieved a **49.9% corpus compression** (129,181 tokens retained from 257,849) while preserving 100% of criminal liability conditions.
- 3. Inverted Index Structure & Stemmer Collisions:** Pipeline A indexes 8,355 unique stemmed terms with coordinate tuples (sent_id, word_offset). We uncovered two fatal bottlenecks: (a) execution vs. executive: Both stem to execut, causing 8 false positives and plunging precision to **38.5%**. (b) suit vs. suitable: Both stem to suit, causing 7 false positives from employment cases and dropping precision to **58.8%**.

■ Lavanya: Observations on Custom Tokenization, ML POS & NER

- 1. Custom Legal Regex:** Cascaded regex patterns preserve statutory citations (Section 302 IPC), constitutional articles (Article 21), rupee amounts (₹50,000/-), and Latin maxims (suo motu) as atomic single tokens, halving token counts from 10 to 5 on penal citations.
- 2. Penn Treebank Syntax Errors:** Default NLTK POS taggers fail on court terms: learned counsel is tagged as a past verb (VBD) instead of adjective (JJ); quashing is tagged as participle (VBG) instead of remedy noun (NN); ad-interim is split into unknown punctuation.
- 3. Supervised ML POS Classifier:** Trained a Multi-Class Logistic Regression model with L2 regularization using contextual n-gram features (suffixes 2-4, prefixes, title case, context words). Achieved **76.7% Test Accuracy** and **0.705 Weighted F1-Score** on held-out court text, successfully predicting learned as JJ (P=0.882) and quashing as NN (P=0.914).
- 4. Legal NER & BPE Subwords:** Extracted 4 statutory entities (LEGAL_SECTION, LEGAL_STATUTE, LEGAL_COURT, LEGAL_CITATION). Trained 6,000-vocabulary BPE subwords segmenting rare words like unconstitutional into un + constitution + al.

■ Raju: Observations on Lemmatization, Dual Index, IR Engine & Evaluation

- 1. Contextual Lemmatization Victory:** WordNet/spaCy lemmatizers guided by POS context preserve execution (noun) and executive (noun/adj) as distinct lemmas. Precision on the 'executive' query jumped from 38.5% in Pipeline A to **100.0% in Pipeline B** with zero false positives. Similarly, suit and suitable are kept distinct (100% precision).
- 2. Dual-Level Indexing (3.5× Speedup):** Engineered a two-layer index (9,005 terms). Layer 1 indexes intact compound phrases ('section 302 ipc'). A query for this citation is a direct \$O(1)\$ hash table lookup, bypassing multi-word coordinate merges. Query latency dropped from 0.14 ms to **0.04 ms (3.5× faster!)**.
- 3. Multi-Mode IR Engine:** Implemented Dijkstra's Shunting-Yard Boolean parser (supporting AND, OR, NOT and quoted phrases) and Vector Space TF-IDF Cosine similarity with length normalization.
- 4. 15-Query Controlled Evaluation:** Pipeline B achieved **0.7721 Precision** (+8.9% over Pipeline A), **0.7979 F1-Score** (+5.7%), **0.8111 Precision@5** (+9.0%), and **0.04 ms latency**, while retaining 100% of the recall (0.8904). Pipeline B conclusively wins across all dimensions.

PART 2: 25 COMPREHENSIVE VIVA QUESTIONS & MODEL ANSWERS

Q1 (For Brunda): Why did you choose Conservative Cleaning over Aggressive Cleaning?

Model Answer: Aggressive cleaning strips all non-alphanumerics. In Indian case law, this is disastrous: ₹50,000/- becomes '50 000' (erasing the rupee sign and splitting the number), and Section 376(2)(i) IPC becomes 'section 376 2 i ipc' (deleting parentheses that define the specific aggravated child rape clause under the POCSO Act harmonization). Conservative cleaning normalizes line-break hyphenations while strictly preserving statutory brackets, currency symbols, and quotes.

Q2 (For Brunda): What defect occurs when standard NLTK Word Tokenizer is used on legal citations?

Model Answer: NLTK uses Penn Treebank regular expressions, treating periods, slashes, and spaces within citations as general word boundaries. It fragments 'Section 302 IPC' into ['Section', '302', 'IPC']. In downstream retrieval, a search for Section 302 will match any document containing 'Section' or '302' unless expensive positional coordinate merges are performed.

Q3 (For Brunda): Why did you reject the Lancaster Stemmer in favor of Porter?

Model Answer: Lancaster uses hyper-aggressive, iterative replacement rules that mutilate legal morphology: 'legal' -> 'leg' (conflating law with the human body), 'custody' -> 'cust', 'bailment' -> 'bail' (conflating civil contracts under Indian Contract Act Section 148 with criminal surety bail), and 'appellants' -> 'appl'. Porter uses calibrated 5-step rules that preserve basic roots.

Q4 (For Brunda): If Snowball and Porter produce virtually identical stems, why pick Porter for Pipeline A?

Model Answer: On our 257k legal tokens, Porter and Snowball produce 98.4% identical roots. Crucially, Snowball fails on the exact same legal polysemes as Porter—both collapse execution and executive to 'execut', and suit and suitable to 'suit'. Porter (1980) was chosen because it is the canonical, universally recognized baseline in IR literature (Manning et al., 2008), providing a recognized standard proving that suffix stripping itself is fundamentally inadequate.

Q5 (For Brunda): How did you handle OCR hyphenation errors across line wraps in court PDFs?

Model Answer: Digital court PDFs often break words across line boundaries with hyphens (e.g. 'crimi-\\nnal'). In our conservative normalization pass, we used regular expression lookarounds to detect alphabetic tokens split by a hyphen and a newline character, recombining them into their intact single word ('criminal'). This prevented artificial vocabulary inflation and preserved token continuity.

Q6 (For Shreerenu): Why can't we use standard English stopword lists in criminal jurisprudence?

Model Answer: Standard stopword lists remove negation words like not, no, without, unless, and against. In criminal law, stripping these words causes Catastrophic Liability Inversion: 'applicant is not guilty' becomes ['applicant', 'guilty'], and 'no grounds for detention' becomes ['grounds', 'detention']. A search for acquittal retrieves convictions! Precision collapses to 0% on negative condition queries.

Q7 (For Shreerenu): What is your Protected Stopword Policy and what compression did it achieve?

Model Answer: Our policy whitelists 8 critical polarity and conditional operators: not, no, never, without, unless, except, until, against. Non-semantic filler words like 'the', 'is', 'at' are filtered out. Across our corpus, this reduced raw tokens from 257,849 to 129,181, achieving a 49.9% corpus compression while preserving 100% of criminal polarity conditions.

Q8 (For Shreerenu): Explain the data structure of your Word-Level Positional Inverted Index in Pipeline A.

Model Answer: In Pipeline A, the index is a hash map where each key is a stemmed vocabulary term (8,355 unique stems). The value is a postings dictionary mapping each document ID to a list of coordinate tuples: (sentence_index, word_offset). For example, 'execut' maps to {'D08': [(14, 3), (42, 11)], 'D24': [(112, 7)]}. This allows verifying whether two stemmed words appear in adjacent positions.

Q9 (For Shreerenu): Explain the over-stemming collision in Pipeline A with the 'executive' query.

Model Answer: The word 'execution' refers to decree enforcement under the CPC, while 'executive' refers to executive magistrates under the CrPC. Porter truncates both words to 'execut'. When a user searches for 'executive', Pipeline A loads postings for 'execut' and retrieves 13 judgments. Only 5 are true executive magistrate cases; 8 are civil decree executions. Precision collapses to 38.5%.

Q10 (For Shreerenu): Explain the second over-stemming collision in Pipeline A: 'suit' vs. 'suitable'.

Model Answer: The legal noun 'suit' refers to civil title or partition lawsuits under the CPC, while 'suitable' is an ordinary English adjective used in employment or service jurisprudence. Porter strips both words to 'suit'. A search for 'suit' retrieves 17 judgments, 7 of which are employment or service disputes having nothing to do with civil suits, plunging precision to 58.8%.

Q11 (For Lavanya): How does your Custom Legal Regex Tokenizer prevent citation fragmentation?

Model Answer: Our tokenizer uses prioritized regular expression cascades that match entire statutory citations ('Section 302 IPC'), constitutional articles ('Article 21'), monetary conditions ('₹50,000/-'), and Latin maxims ('suo motu') before falling back to general word matching. For example, 'Bail under Section 438 CrPC of ₹50,000/-' is tokenized into 5 atomic tokens instead of 10 fragmented tokens.

Q12 (For Lavanya): Why does the Penn Treebank POS tagger fail on phrases like 'learned counsel'?

Model Answer: Penn Treebank taggers were trained on the Wall Street Journal, where 'learned' functions as past tense verb 'to learn' (VBN/VBD). In judicial text, 'learned' is an honorific adjective (JJ) modifying counsel or judge. Our custom POS rules and ML classifier correctly classify it as JJ based on its syntactic position before legal nouns.

Q13 (For Lavanya): What features did you engineer for your Supervised ML POS Classifier?

Model Answer: We engineered contextual n-gram features: character suffixes of length 2 to 4 (-ing, -tion, -able, -ed), prefixes (un-, non-), orthographic flags (is_title, is_upper, has_digit, has_hyphen), and preceding/following word tokens. We trained a Multi-Class Logistic Regression model with L2 regularization on Indian court text, achieving 76.7% Test Accuracy and 0.705 Weighted F1-Score.

Q14 (For Lavanya): What domain-specific entities does your Legal NER module extract?

Model Answer: Our Legal NER module extracts four statutory domain entities: LEGAL_SECTION (Section 438 CrPC), LEGAL_STATUTE (NDPS Act, PMLA), LEGAL_COURT (Supreme Court of India), and LEGAL_CITATION (AIR 2020 SC 123). It also extracts standard entities: PERSON, ORG, MONEY, and GPE.

Q15 (For Lavanya): Why did you train a Byte Pair Encoding (BPE) model with a 6,000 vocabulary?

Model Answer: Legal text contains complex morphological compounds, Latin prefixes, and hyphenated terms. Traditional word tokenizers replace unseen words with . Our 6,000-vocabulary BPE model iteratively merges frequent character pairs, allowing rare words like 'unconstitutional' to be segmented into 'un' + 'constitution' + 'al', eliminating out-of-vocabulary loss.

Q16 (For Raju): How does contextual lemmatization in Pipeline B resolve the 'executive' collision?

Model Answer: Unlike stemmers that chop word endings using static suffix rules, WordNet and spaCy lemmatizers check morphological dictionaries conditioned on part-of-speech tags. Because our ML POS tagger tags 'execution' as a Noun and 'executive' as an Adjective/Noun, lemmatization preserves 'execution' and 'executive' as distinct dictionary lemmas. Querying 'executive' retrieves only the 5 true cases, achieving 100% precision with zero false positives.

Q17 (For Raju): Explain the Dual-Level Atomic Positional Index and why it is 3.5× faster.

Model Answer: In Pipeline A, searching for 'Section 302 IPC' requires loading three separate postings lists ('section', '302', 'ipc') and running an iterative coordinate merge checking if pos(302) == pos(section) + 1. In Pipeline B, the index has two layers: Layer 1 indexes intact compound phrases ('section 302 ipc') directly. A search for this citation is a single O(1) hash table lookup. Layer 2 indexes constituent lemmas for single-word queries. This direct lookup reduced query latency from 0.14 ms to 0.04 ms (3.5× faster).

Q18 (For Raju): How does your Boolean Retrieval Engine parse queries like "'anticipatory bail' AND NOT murder"?

Model Answer: We implemented Dijkstra's Shunting-Yard algorithm to convert infix Boolean queries into Reverse Polish Notation (postfix). Operator precedence is enforced: NOT > AND > OR. Quoted phrases are treated as atomic tokens. The engine evaluates the postfix stack using set operations: it computes the intersection of postings for 'anticipatory bail' and takes the set difference with postings for 'murder'.

Q19 (For Raju): Explain the mathematical formula for your Vector Space TF-IDF Cosine Retrieval.

Model Answer: We compute sublinear term frequency: $TF(t,d) = 1 + \log(TF_t,d)$ for $TF > 0$. The inverse document frequency is $IDF(t) = \log(N / DF_t)$, where $N=25$. The query-document score is the cosine angle normalized against Euclidean document length: $Score(q,d) = (\sum TF-IDF(t,d) * TF-IDF(t,q)) / (\|d\| * \|q\|)$. Length normalization eliminates bias toward lengthy High Court judgments.

Q20 (For Raju): Walk us through the 15-query benchmark evaluation. Why is Pipeline B the winning model?

Model Answer: Across 15 legal benchmark queries, Pipeline B achieved Mean Precision of 0.7721 vs. 0.7089 (+8.9% gain), Mean Recall of 0.8904 vs. 0.8904 (100% recall retained), Mean F1-Score of 0.7979 vs. 0.7549 (+5.7% harmonic gain), Precision@5 of 0.8111 vs. 0.7444 (+9.0%), and Latency of 0.04 ms vs. 0.14 ms (3.5× faster). Pipeline B decisively wins because it eliminates false positives without sacrificing a single point of recall.

Q21 (All Members): Why did Pipeline B improve Precision WITHOUT dropping Recall?

Model Answer: In traditional IR, precision is boosted by narrowing search criteria, which often drops recall by excluding borderline cases. In our legal domain, however, Pipeline A's low precision was caused by artificial false positives from stemmer collisions ('execut' matching decree cases) and fragmented citations. Pipeline B eliminated linguistic false positives through POS-aware lemmatization and atomic citation preservation while retaining all true relevant documents, thereby preserving 100% of the recall (0.8904).

Q22 (All Members): How would your system scale if the corpus grew from 25 judgments to 100,000 judgments?

Model Answer: Currently, our postings lists reside in in-memory Python dictionaries with JSON persistence. At 100,000 documents: (1) We would migrate from in-memory hash maps to disk-based inverted indexes using Block Sort-Based Indexing (BSBI) or Single-Pass In-Memory Indexing (SPIMI). (2) We would compress postings lists using Variable Byte (Varint) or Elias-Gamma gap encoding. (3) Layer 1 atomic compound indexing would be limited to statutory entities extracted by NER to prevent vocabulary explosion.

Q23 (All Members): What is the significance of the Forensic IR Execution Trace in your Streamlit application?

Model Answer: In legal informatics, search engines cannot be black boxes. When a judge or advocate searches case law, they must verify why a case was retrieved. Our forensic execution trace logs the raw query, tokenization output, matching postings coordinates, and classifies retrieved documents into True Positives, False Positives, and False Negatives against ground truth, providing academic transparency.

Q24 (All Members): How does your system handle multilingual or vernacular terms common in Indian court judgments?

Model Answer: Indian court judgments frequently incorporate vernacular legal terms from Hindi, Urdu, and regional languages (e.g., panchnama, challan, kabala, talak, vakalatnama). In our pipeline, conservative cleaning preserves non-ASCII tokens, and our 6,000-vocabulary BPE subword model segments unseen vernacular constructs into shared byte units, preventing out-of-vocabulary crashes.

Q25 (All Members): What is the single most important lesson from this project regarding generic vs. domain-specific NLP?

Model Answer: Generic NLP pipelines make assumptions suited for news and social media—such as aggressive stopword removal and greedy suffix stripping. In domain-specific legal text, text normalization is inseparable from statutory reasoning. Protecting statutory citations as atomic units, shielding criminal negations, and using POS-guided lemmatization is not an optional optimization—it is essential for legal justice.